



BROADCAST VARIABLES

Sending Read-Only Data to Every Executor — Once

Broadcast Join

Read-Only

Executor Cache

Threshold

Interview Ready in 6 Slides

Swipe to start learning →

P THE NAIVE PROBLEM

Without broadcasting, a variable referenced inside a closure (a lookup dict, a small table) gets **serialized and re-shipped with every single Task**.

Why That's Wasteful

1

A Job runs **10,000 Tasks** across 50 Executors



2

Naively, the same 50MB lookup table is serialized **10,000 times** — once per Task



3

Network + serialization overhead scales with **Task count**, not data size



4

Broadcast fixes this: ship the data **once per Executor**, not once per Task

Interview Tip: Think of it as replacing 10,000 copies with 50 — one per Executor, not one per Task.

H BITTORRENT-STYLE DISTRIBUTION

Spark doesn't push the broadcast from a single source — it spreads it peer-to-peer, like a torrent.

From Driver to Every Executor

1 Driver serializes the variable once and splits it into **blocks**



2 Executors pull blocks from the Driver **and from each other**



3 Each Executor caches one full, deserialized copy in memory



4 Every Task on that Executor reads the **same in-memory copy** — zero extra shipping

Interview Tip: This peer-to-peer spread is why broadcasting scales well even to hundreds of Executors.

J THE MOST COMMON USE CASE

RELATED APIS

`sc.broadcast()``broadcast(df) hint``autoBroadcastJoinThreshold`

Turning a Shuffle Into a Lookup

1 Joining a huge fact table with a small dimension table



2 Catalyst broadcasts the small side instead of shuffling both sides



3 The join becomes a local, **in-memory hash lookup** — no shuffle at all



4 Controlled by `spark.sql.autoBroadcastJoinThreshold` (default 10MB)

Interview Tip: Force it explicitly with `broadcast(df)` when Catalyst's size estimate is wrong — e.g. after heavy upstream filtering.

W THE LIMITS

Broadcast variables are **read-only** and must fit comfortably in every Executor's memory — pushing either limit causes failures, not slowdowns.

Four Ways It Goes Wrong

1 Broadcasting past the threshold (or forcing a large table) can **OOM Executors**



2 Broadcast variables **cannot be updated** after creation — no mutation, ever



3 Very large broadcasts slow **Job startup** — every Executor waits for the full copy



4 Rule of thumb: broadcast tables under a few hundred MB, not GB-scale tables

Interview Tip: An OOM right at Job start, before any Task output appears, is a classic sign of an oversized broadcast.

Save this slide before your next Data Engineering interview

CONCEPT	PURPOSE	KEY INTERVIEW FACT
Broadcast Variable	Ship read-only data once	One copy per Executor, not per Task
Broadcast Join	Avoid shuffle	Small table replicated, hash-joined locally
autoBroadcastJoinThreshold	Controls auto broadcast	Default 10MB
broadcast(df) hint	Manual override	Use when Catalyst's estimate is wrong

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep